# Introduction to MLIR

## Session 2, Part 1 — Linalg and Tiling, CPU

SC26 Tutorial

# Setup

- **It is a fresh runtime.** Nothing from Session 1 carries over, so the notebook downloads the same toolchain again. Any runtime will do: this part runs on the CPU.

# Setup

- **It is a fresh runtime.** Nothing from Session 1 carries over, so the notebook downloads the same toolchain again. Any runtime will do: this part runs on the CPU.
- **The exercise edits one MLIR file, a schedule.** No C++ and no rebuild in this session: edit, save, re-run the cell.

# Setup

- **It is a fresh runtime.** Nothing from Session 1 carries over, so the notebook downloads the same toolchain again. Any runtime will do: this part runs on the CPU.
- **The exercise edits one MLIR file, a schedule.** No C++ and no rebuild in this session: edit, save, re-run the cell.
- **One exercise:** tile matmul's third loop. Steps 1 and 3 we read and run together.

# Setup

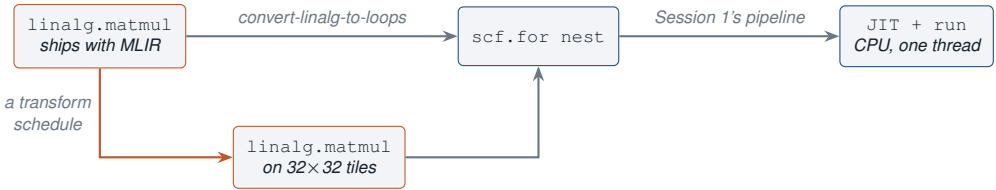
- **It is a fresh runtime.** Nothing from Session 1 carries over, so the notebook downloads the same toolchain again. Any runtime will do: this part runs on the CPU.
- **The exercise edits one MLIR file, a schedule.** No C++ and no rebuild in this session: edit, save, re-run the cell.
- **One exercise:** tile matmul's third loop. Steps 1 and 3 we read and run together.
- **Colab trick:** `Ctrl+F8` runs every cell above the current one, to catch up after losing track.

# One Level Higher: Linalg Dialect



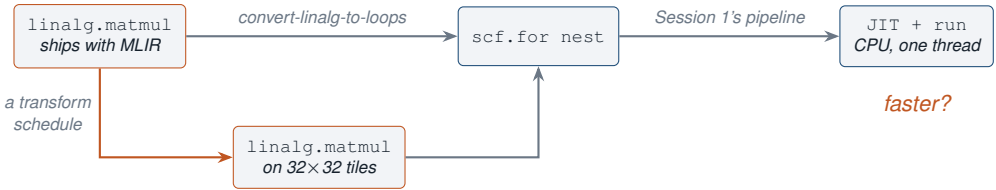

- Session 1 defined `blas.matmul` and wrote its lowering. Linalg is a dialect that ships with MLIR: its ops already cover matmul and matvec, and their lowering is already written.
- Linalg is a more general representation of linear-algebra computations — specifically einsum computations, built on polyhedral compilation methods.

# One Level Higher: Linalg Dialect




- Session 1 defined `blas.matmul` and wrote its lowering. Linalg is a dialect that ships with MLIR: its ops already cover matmul and matvec, and their lowering is already written.
- Linalg is a more general representation of linear-algebra computations — specifically einsum computations, built on polyhedral compilation methods.
- Linalg supports general loop transformations, like tiling.

# One Level Higher: Linalg Dialect



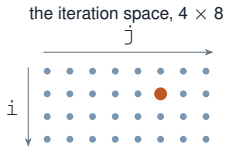

- Session 1 defined `blas.matmul` and wrote its lowering. Linalg is a dialect that ships with MLIR: its ops already cover matmul and matvec, and their lowering is already written.
- Linalg is a more general representation of linear-algebra computations — specifically einsum computations, built on polyhedral compilation methods.
- Linalg supports general loop transformations, like tiling.
- Same computation, same single thread: does it get faster?

# Step 1 — Linalg Ops

# Linalg Primer: Modeling a Loop Nest

A bias add: `c[i, j] = a[i, j] + b[j]`

```
for i in range(4):
  for j in range(8):
    c[i, j] = a[i, j] + b[j]
```

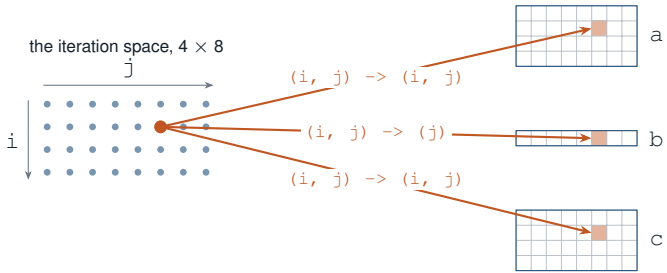



- The loops span an iteration space: one point per `(i, j)`.

# Linalg Primer: Modeling a Loop Nest

A bias add: `c[i, j] = a[i, j] + b[j]`

```
for i in range(4):
  for j in range(8):
    c[i, j] = a[i, j] + b[j]
```

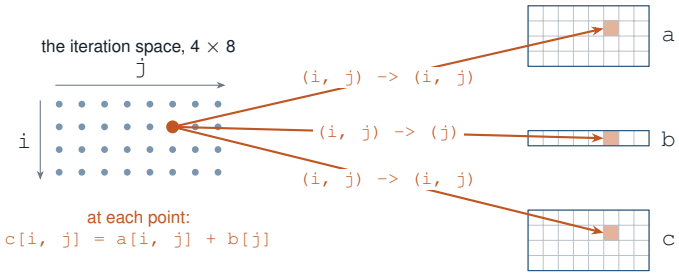



- The loops span an iteration space: one point per `(i, j)`.
- Each access is a map from that space to the data: the point `(i, j)` reads `a[i, j]` and `b[j]`, and writes `c[i, j]`.

# Linalg Primer: Modeling a Loop Nest

A bias add: `c[i, j] = a[i, j] + b[j]`

```
for i in range(4):
  for j in range(8):
    c[i, j] = a[i, j] + b[j]
```




- The loops span an iteration space: one point per `(i, j)`.
- Each access is a map from that space to the data: the point `(i, j)` reads `a[i, j]` and `b[j]`, and writes `c[i, j]`.
- Each point runs the loop body once.

# Linalg Primer: From Loops to `linalg.generic`

### the loop nest

```
scf.for %i = %c0 to %c4 step %c1 {
  scf.for %j = %c0 to %c8 step %c1 {
    %x = memref.load %a[%i, %j] : memref<4x8xf32>
    %y = memref.load %b[%j] : memref<8xf32>
    %s = arith.addf %x, %y : f32
    memref.store %s, %c[%i, %j] : memref<4x8xf32>
  }
}
```

### the same, as `linalg.generic`

```
#id = affine_map<(i, j) -> (i, j)>
#bcast = affine_map<(i, j) -> (j)>
linalg.generic {
    indexing_maps = [#id, #bcast, #id],
    iterator_types = ["parallel", "parallel"]}
    ins(%a, %b : memref<4x8xf32>, memref<8xf32>)
    outs(%c : memref<4x8xf32>) {
^bb0(%x: f32, %y: f32, %out: f32):
  %s = arith.addf %x, %y : f32
  linalg.yield %s : f32
}
```

# Linalg Primer: From Loops to `linalg.generic`

**the loop nest**

```
scf.for %i = %c0 to %c4 step %c1 {
  scf.for %j = %c0 to %c8 step %c1 {
    %x = memref.load %a[%i, %j] : memref<4x8xf32>
    %y = memref.load %b[%j] : memref<8xf32>
    %s = arith.addf %x, %y : f32
    memref.store %s, %c[%i, %j] : memref<4x8xf32>
  }
}
```

**the same, as `linalg.generic`**

```
#id = affine_map<(i, j) -> (i, j)>
#bcast = affine_map<(i, j) -> (j)>
linalg.generic {
    indexing_maps = [#id, #bcast, #id],
    iterator_types = ["parallel", "parallel"]}
    ins(%a, %b : memref<4x8xf32>, memref<8xf32>)
    outs(%c : memref<4x8xf32>) {
^bb0(%x: f32, %y: f32, %out: f32):
  %s = arith.addf %x, %y : f32
  linalg.yield %s : f32
}
```

- `iterator_types`: one per loop, and the dependence between points. Both are `parallel`: no point depends on another.

# Linalg Primer: From Loops to `linalg.generic`

**the loop nest**

```
scf.for %i = %c0 to %c4 step %c1 {
  scf.for %j = %c0 to %c8 step %c1 {
    %x = memref.load %a[%i, %j] : memref<4x8xf32>
    %y = memref.load %b[%j] : memref<8xf32>
    %s = arith.addf %x, %y : f32
    memref.store %s, %c[%i, %j] : memref<4x8xf32>
  }
}
```

**the same, as `linalg.generic`**

```
#id = affine_map<(i, j) -> (i, j)>
#bcast = affine_map<(i, j) -> (j)>
linalg.generic {
    indexing_maps = [#id, #bcast, #id],
    iterator_types = ["parallel", "parallel"]}
    ins(%a, %b : memref<4x8xf32>, memref<8xf32>)
    outs(%c : memref<4x8xf32>) {
^bb0(%x: f32, %y: f32, %out: f32):
  %s = arith.addf %x, %y : f32
  linalg.yield %s : f32
}
```

- `iterator_types`: one per loop, and the dependence between points. Both are `parallel`: no point depends on another.
- `indexing_maps`: one per operand, the maps from the last slide.

# Linalg Primer: From Loops to `linalg.generic`

**the loop nest**

```
scf.for %i = %c0 to %c4 step %c1 {
  scf.for %j = %c0 to %c8 step %c1 {
    %x = memref.load %a[%i, %j] : memref<4x8xf32>
    %y = memref.load %b[%j] : memref<8xf32>
    %s = arith.addf %x, %y : f32
    memref.store %s, %c[%i, %j] : memref<4x8xf32>
  }
}
```

**the same, as `linalg.generic`**

```
#id = affine_map<(i, j) -> (i, j)>
#bcast = affine_map<(i, j) -> (j)>
linalg.generic {
    indexing_maps = [#id, #bcast, #id],
    iterator_types = ["parallel", "parallel"]}
    ins(%a, %b : memref<4x8xf32>, memref<8xf32>)
    outs(%c : memref<4x8xf32>) {
^bb0(%x: f32, %y: f32, %out: f32):
  %s = arith.addf %x, %y : f32
  linalg.yield %s : f32
}
```

- `iterator_types`: one per loop, and the dependence between points. Both are `parallel`: no point depends on another.
- `indexing_maps`: one per operand, the maps from the last slide.
- The region is the loop body, run at each point: it receives the elements and yields the result.

# Linalg Primer: From Loops to `linalg.generic`

**the loop nest**

```
scf.for %i = %c0 to %c4 step %c1 {
  scf.for %j = %c0 to %c8 step %c1 {
    %x = memref.load %a[%i, %j] : memref<4x8xf32>
    %y = memref.load %b[%j] : memref<8xf32>
    %s = arith.addf %x, %y : f32
    memref.store %s, %c[%i, %j] : memref<4x8xf32>
  }
}
```

**the same, as `linalg.generic`**

```
#id = affine_map<(i, j) -> (i, j)>
#bcast = affine_map<(i, j) -> (j)>
linalg.generic {
    indexing_maps = [#id, #bcast, #id],
    iterator_types = ["parallel", "parallel"]}
    ins(%a, %b : memref<4x8xf32>, memref<8xf32>)
    outs(%c : memref<4x8xf32>) {
^bb0(%x: f32, %y: f32, %out: f32):
  %s = arith.addf %x, %y : f32
  linalg.yield %s : f32
}
```

- `iterator_types`: one per loop, and the dependence between points. Both are `parallel`: no point depends on another.
- `indexing_maps`: one per operand, the maps from the last slide.
- The region is the loop body, run at each point: it receives the elements and yields the result.
- `ins` are read; `outs` is written. The region also receives the output's current value, `%out`, so an op can accumulate into it.

# Named Linalg Ops

In `linalg_matmul.mlir` and `linalg_matvec.mlir`:

```
linalg.matmul ins(%a, %b : ...) outs(%c : ...)
linalg.matvec ins(%mat, %vec : ...) outs(%out : ...)
```

# Named Linalg Ops

In `linalg_matmul.mlir` and `linalg_matvec.mlir`:

```
linalg.matmul ins(%a, %b : ...) outs(%c : ...)
linalg.matvec ins(%mat, %vec : ...) outs(%out : ...)
```

- A named Linalg op is really a `linalg.generic` whose indexing maps, iterator types and region are fixed by its name:

| | `linalg.matmul` | `linalg.matvec` |
|---|---|---|
| indexing_maps | `a: (i, j, k) -> (i, k)`<br>`b: (i, j, k) -> (k, j)`<br>`c: (i, j, k) -> (i, j)` | `mat: (i, j) -> (i, j)`<br>`vec: (i, j) -> (j)`<br>`out: (i, j) -> (i)` |
| iterator_types | `parallel, parallel, reduction` | `parallel, reduction` |
| region yields | `c + a * b` | `out + mat * vec` |

# Named Linalg Ops

In `linalg_matmul.mlir` and `linalg_matvec.mlir`:

```
linalg.matmul ins(%a, %b : ...) outs(%c : ...)
linalg.matvec ins(%mat, %vec : ...) outs(%out : ...)
```

- A named Linalg op is really a `linalg.generic` whose indexing maps, iterator types and region are fixed by its name:

| | `linalg.matmul` | `linalg.matvec` |
|---|---|---|
| `indexing_maps` | `a: (i, j, k) -> (i, k)`<br>`b: (i, j, k) -> (k, j)`<br>`c: (i, j, k) -> (i, j)` | `mat: (i, j) -> (i, j)`<br>`vec: (i, j) -> (j)`<br>`out: (i, j) -> (i)` |
| `iterator_types` | `parallel, parallel, reduction` | `parallel, reduction` |
| region yields | `c + a * b` | `out + mat * vec` |

- Running `linalg-generalize-named-ops` rewrites each named op as the `linalg.generic` it stands for: that is where this table comes from.

# Named Linalg Ops

In `linalg_matmul.mlir` and `linalg_matvec.mlir`:

```
linalg.matmul ins(%a, %b : ...) outs(%c : ...)
linalg.matvec ins(%mat, %vec : ...) outs(%out : ...)
```

- A named Linalg op is really a `linalg.generic` whose indexing maps, iterator types and region are fixed by its name:

| | `linalg.matmul` | `linalg.matvec` |
|---|---|---|
| `indexing_maps` | `a: (i, j, k) -> (i, k)`<br>`b: (i, j, k) -> (k, j)`<br>`c: (i, j, k) -> (i, j)` | `mat: (i, j) -> (i, j)`<br>`vec: (i, j) -> (j)`<br>`out: (i, j) -> (i)` |
| `iterator_types` | `parallel, parallel, reduction` | `parallel, reduction` |
| region yields | `c + a * b` | `out + mat * vec` |

- Running `linalg-generalize-named-ops` rewrites each named op as the `linalg.generic` it stands for: that is where this table comes from.
- All Linalg named ops and `linalg.generic` implement the `LinalgOp` interface, and Linalg transformations are written against that interface, so any transformation applies to all of them.

# Lowered to Loops

`mlir-opt linalg_matvec.mlir --convert-linalg-to-loops`, the body of `@matvec`:

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index
%c512 = arith.constant 512 : index
scf.for %arg3 = %c0 to %c128 step %c1 {                    // i
  scf.for %arg4 = %c0 to %c512 step %c1 {                  // j
    %0 = memref.load %arg0[%arg3, %arg4] : memref<128x512xf32>
    %1 = memref.load %arg1[%arg4] : memref<512xf32>
    %2 = memref.load %arg2[%arg3] : memref<128xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    memref.store %4, %arg2[%arg3] : memref<128xf32>
  }
}
```

# Lowered to Loops

`mlir-opt linalg_matvec.mlir --convert-linalg-to-loops`, the body of `@matvec`:

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index
%c512 = arith.constant 512 : index
scf.for %arg3 = %c0 to %c128 step %c1 {                          // i
  scf.for %arg4 = %c0 to %c512 step %c1 {                        // j
    %0 = memref.load %arg0[%arg3, %arg4] : memref<128x512xf32>
    %1 = memref.load %arg1[%arg4] : memref<512xf32>
    %2 = memref.load %arg2[%arg3] : memref<128xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    memref.store %4, %arg2[%arg3] : memref<128xf32>
  }
}
```

- One `scf.for` per iterator type: `i`, `j`.

# Lowered to Loops

`mlir-opt linalg_matvec.mlir --convert-linalg-to-loops`, the body of `@matvec`:

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index
%c512 = arith.constant 512 : index
scf.for %arg3 = %c0 to %c128 step %c1 {                    // i
  scf.for %arg4 = %c0 to %c512 step %c1 {                  // j
    %0 = memref.load %arg0[%arg3, %arg4] : memref<128x512xf32>
    %1 = memref.load %arg1[%arg4] : memref<512xf32>
    %2 = memref.load %arg2[%arg3] : memref<128xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    memref.store %4, %arg2[%arg3] : memref<128xf32>
  }
}
```

- One `scf.for` per iterator type: `i`, `j`.
- A load of each input, at the indices its map selects.

# Lowered to Loops

`mlir-opt linalg_matvec.mlir --convert-linalg-to-loops`, the body of `@matvec`:

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index
%c512 = arith.constant 512 : index
scf.for %arg3 = %c0 to %c128 step %c1 {                    // i
  scf.for %arg4 = %c0 to %c512 step %c1 {                  // j
    %0 = memref.load %arg0[%arg3, %arg4] : memref<128x512xf32>
    %1 = memref.load %arg1[%arg4] : memref<512xf32>
    %2 = memref.load %arg2[%arg3] : memref<128xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    memref.store %4, %arg2[%arg3] : memref<128xf32>
  }
}
```

- One `scf.for` per iterator type: `i`, `j`.
- A load of each input, at the indices its map selects.
- A load of `out` too: `j` is a reduction, so every point along `j` adds into the same `out[i]`, and the body reads its current value first.

# Lowered to Loops

`mlir-opt linalg_matvec.mlir --convert-linalg-to-loops`, the body of `@matvec`:

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index
%c512 = arith.constant 512 : index
scf.for %arg3 = %c0 to %c128 step %c1 {                          // i
  scf.for %arg4 = %c0 to %c512 step %c1 {                        // j
    %0 = memref.load %arg0[%arg3, %arg4] : memref<128x512xf32>
    %1 = memref.load %arg1[%arg4] : memref<512xf32>
    %2 = memref.load %arg2[%arg3] : memref<128xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    memref.store %4, %arg2[%arg3] : memref<128xf32>
  }
}
```

- One `scf.for` per iterator type: `i`, `j`.
- A load of each input, at the indices its map selects.
- A load of `out` too: `j` is a reduction, so every point along `j` adds into the same `out[i]`, and the body reads its current value first.
- The body, then a store back into `out`. It is Session 1's `matvec_scf.mlir`, generated.

# Lowering and Execution

`convert-linalg-to-loops`
*then Session 1's CPU lowering pipeline*

# Lowering and Execution

**`convert-linalg-to-loops`**
*then Session 1's CPU lowering pipeline*

- `convert-linalg-to-loops` does the job `convert-blas-to-scf` did in Session 1.

# Lowering and Execution

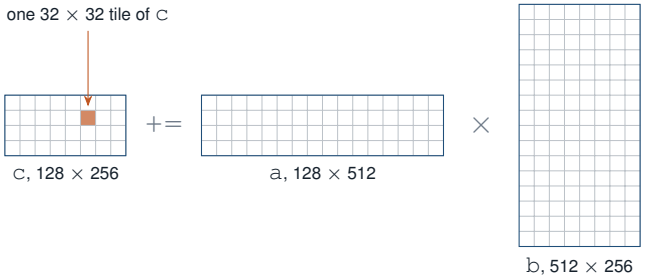
`convert-linalg-to-loops`
*then Session 1's CPU lowering pipeline*

- `convert-linalg-to-loops` does the job `convert-blas-to-scf` did in Session 1.
- JIT-compiled, and checked against numpy's `a @ b`.

OK

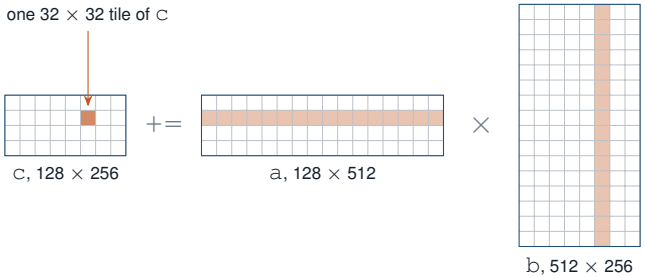
# Step 2 — Tiling, with a Transform Schedule

# Tiling



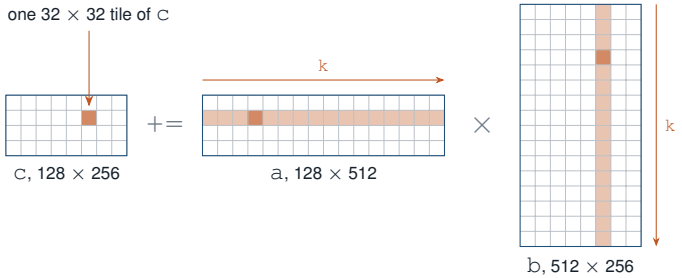

- Tiling splits a loop nest into two sets of loops:
  - Outer loops that iterate over tiles;
  - Inner loops that do the computation within a tile;
  - The same computation, in a different order.

# Tiling




- Tiling splits a loop nest into two sets of loops:
  - Outer loops that iterate over tiles;
  - Inner loops that do the computation within a tile;
  - The same computation, in a different order.
- Tile `i` and `j` by 32, and each 32 × 32 tile of `c` reads a 32-row strip of `a` and a 32-column strip of `b`.

# Tiling




- Tiling splits a loop nest into two sets of loops:
  - Outer loops that iterate over tiles;
  - Inner loops that do the computation within a tile;
  - The same computation, in a different order.
- Tile `i` and `j` by 32, and each 32 × 32 tile of `c` reads a 32-row strip of `a` and a 32-column strip of `b`.
- Tile `k` by 32 too, and each strip is read one 32 × 32 tile at a time: 16 steps along `k` for each tile of `c`.

# The Tile Is a `linalg.matmul`

### the op we started with

```
linalg.matmul
    ins(%a, %b : memref<128x512xf32>,
                 memref<512x256xf32>)
    outs(%c : memref<128x256xf32>)
```

### tiled: the shape of it

```
scf.for %i = 0 to 128 step 32 {                // tile i
  scf.for %j = 0 to 256 step 32 {              // tile j
    scf.for %k = 0 to 512 step 32 {            // tile k
      %a_tile = memref.subview %a[%i, %k] [32, 32]
      %b_tile = memref.subview %b[%k, %j] [32, 32]
      %c_tile = memref.subview %c[%i, %j] [32, 32]
      linalg.matmul ins(%a_tile, %b_tile) outs(%c_tile)
    }
  }
}
```

# The Tile Is a `linalg.matmul`

**the op we started with**

```
linalg.matmul
    ins(%a, %b : memref<128x512xf32>,
                 memref<512x256xf32>)
    outs(%c : memref<128x256xf32>)
```

**tiled: the shape of it**

```
scf.for %i = 0 to 128 step 32 {                  // tile i
  scf.for %j = 0 to 256 step 32 {                // tile j
    scf.for %k = 0 to 512 step 32 {              // tile k
      %a_tile = memref.subview %a[%i, %k] [32, 32]
      %b_tile = memref.subview %b[%k, %j] [32, 32]
      %c_tile = memref.subview %c[%i, %j] [32, 32]
      linalg.matmul ins(%a_tile, %b_tile) outs(%c_tile)
    }
  }
}
```

- The outer loops are `scf.for`s over `i`, `j` and `k`, each stepping by 32.

# The Tile Is a `linalg.matmul`

**the op we started with**

```
linalg.matmul
    ins(%a, %b : memref<128x512xf32>,
                 memref<512x256xf32>)
    outs(%c : memref<128x256xf32>)
```

**tiled: the shape of it**

```
scf.for %i = 0 to 128 step 32 {                // tile i
  scf.for %j = 0 to 256 step 32 {              // tile j
    scf.for %k = 0 to 512 step 32 {            // tile k
      %a_tile = memref.subview %a[%i, %k] [32, 32]
      %b_tile = memref.subview %b[%k, %j] [32, 32]
      %c_tile = memref.subview %c[%i, %j] [32, 32]
      linalg.matmul ins(%a_tile, %b_tile) outs(%c_tile)
    }
  }
}
```

- The outer loops are `scf.for`s over `i`, `j` and `k`, each stepping by 32.
- `memref.subview` picks out a $32 \times 32$ tile of each operand.

# The Tile Is a `linalg.matmul`

**the op we started with**

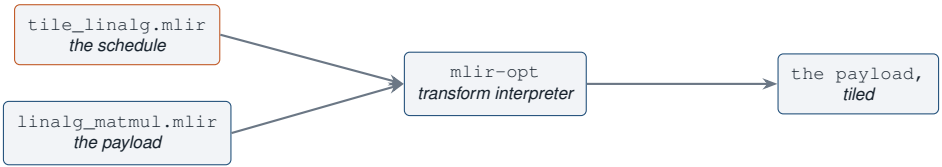
```
linalg.matmul
    ins(%a, %b : memref<128x512xf32>,
                 memref<512x256xf32>)
    outs(%c : memref<128x256xf32>)
```

**tiled: the shape of it**

```
scf.for %i = 0 to 128 step 32 {                // tile i
  scf.for %j = 0 to 256 step 32 {              // tile j
    scf.for %k = 0 to 512 step 32 {            // tile k
      %a_tile = memref.subview %a[%i, %k] [32, 32]
      %b_tile = memref.subview %b[%k, %j] [32, 32]
      %c_tile = memref.subview %c[%i, %j] [32, 32]
      linalg.matmul ins(%a_tile, %b_tile) outs(%c_tile)
    }
  }
}
```

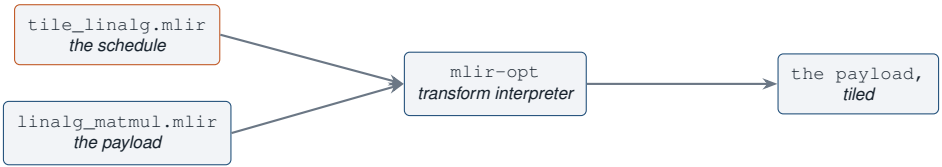
- The outer loops are `scf.for`s over `i`, `j` and `k`, each stepping by 32.
- `memref.subview` picks out a $32 \times 32$ tile of each operand.
- The inner loops are still one `linalg.matmul`: the op we started with, on $32 \times 32$ tiles.

# A Schedule: MLIR That Transforms MLIR



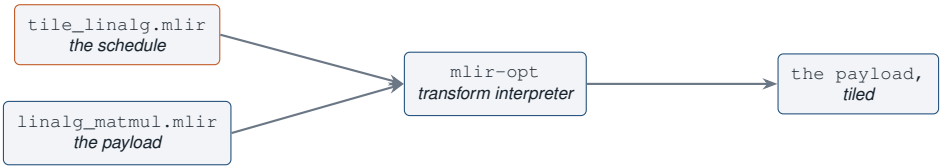

- In Session 1 a rewrite meant C++: a pattern, a plugin build, and reloading `mlir-opt`.

# A Schedule: MLIR That Transforms MLIR



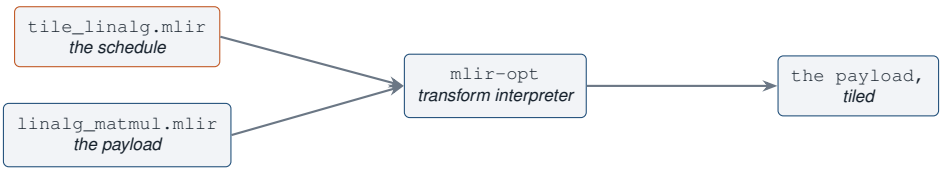

- In Session 1 a rewrite meant C++: a pattern, a plugin build, and reloading `mlir-opt`.
- The transform dialect specifies transformations on operations, and applies them.

# A Schedule: MLIR That Transforms MLIR




- In Session 1 a rewrite meant C++: a pattern, a plugin build, and reloading `mlir-opt`.
- The transform dialect specifies transformations on operations, and applies them.
- Its operations are transformations to apply. Its values are handles to operations, and an operation's results are handles to the operations it creates.

# A Schedule: MLIR That Transforms MLIR




- In Session 1 a rewrite meant C++: a pattern, a plugin build, and reloading `mlir-opt`.
- The transform dialect specifies transformations on operations, and applies them.
- Its operations are transformations to apply. Its values are handles to operations, and an operation's results are handles to the operations it creates.
- No C++, no rebuild: edit the schedule, re-run the cell.

# Reference: `tile_linalg.mlir`, Step 1 of 3 — Schedule

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    transform.yield
  }
}
```

# Reference: `tile_linalg.mlir`, Step 1 of 3 — Schedule

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    transform.yield
  }
}
```

- The interpreter calls `@__transform_main`, passing the payload's top-level op as `%root`.

# Reference: `tile_linalg.mlir`, Step 1 of 3 — Schedule

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    transform.yield
  }
}
```

- The interpreter calls `@__transform_main`, passing the payload's top-level op as `%root`.
- `match` finds ops in the payload — by interface, not by name, so any Linalg op. `%ops` is a handle: a reference to payload ops, not a value in the payload program.

# Reference: `tile_linalg.mlir`, Step 1 of 3 — Schedule

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    transform.yield
  }
}
```

- The interpreter calls `@__transform_main`, passing the payload's top-level op as `%root`.
- `match` finds ops in the payload — by interface, not by name, so any Linalg op. `%ops` is a handle: a reference to payload ops, not a value in the payload program.
- `tile_using_for` takes one tile size per loop, in `iterator_types` order. `[32, 32]` tiles the first two.

# Reference: `tile_linalg.mlir`, Step 1 of 3 — Schedule

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    transform.yield
  }
}
```

- The interpreter calls `@__transform_main`, passing the payload's top-level op as `%root`.
- `match` finds ops in the payload — by interface, not by name, so any Linalg op. `%ops` is a handle: a reference to payload ops, not a value in the payload program.
- `tile_using_for` takes one tile size per loop, in `iterator_types` order. `[32, 32]` tiles the first two.
- It returns the tiled op, then one handle per loop it created: `%loops:2`.

# Reference: `tile_linalg.mlir`, Step 2 of 3 — Applied to Matvec

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c512 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    %subview_2 = memref.subview %arg1[%arg4] [32] [1] : ... to memref<32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3] [32] [1] : ... to memref<32xf32, ...>
    linalg.matvec ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

# Reference: `tile_linalg.mlir`, Step 2 of 3 — Applied to Matvec

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c512 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    %subview_2 = memref.subview %arg1[%arg4] [32] [1] : ... to memref<32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3] [32] [1] : ... to memref<32xf32, ...>
    linalg.matvec ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- Two `scf.for` tile loops stepping by 32: both of matvec's loops are tiled.

# Reference: `tile_linalg.mlir`, Step 2 of 3 — Applied to Matvec

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c512 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    %subview_2 = memref.subview %arg1[%arg4] [32] [1] : ... to memref<32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3] [32] [1] : ... to memref<32xf32, ...>
    linalg.matvec ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- Two `scf.for` tile loops stepping by 32: both of matvec's loops are tiled.
- `memref.subview` takes a 32-wide view of each operand: offsets, sizes, strides. Nothing is copied.

# Reference: `tile_linalg.mlir`, Step 2 of 3 — Applied to Matvec

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c512 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    %subview_2 = memref.subview %arg1[%arg4] [32] [1] : ... to memref<32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3] [32] [1] : ... to memref<32xf32, ...>
    linalg.matvec ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- Two `scf.for` tile loops stepping by 32: both of matvec's loops are tiled.
- `memref.subview` takes a 32-wide view of each operand: offsets, sizes, strides. Nothing is copied.
- Each slice follows its operand's indexing map. `vec`'s is `(i, j) -> (j)`, so its slice moves with the `j` loop (`%arg4`); `out`'s is `(i, j) -> (i)`, so it moves with the `i` loop (`%arg3`). `mat`'s is the identity: it moves with both.

# Reference: `tile_linalg.mlir`, Step 2 of 3 — Applied to Matvec

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c512 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    %subview_2 = memref.subview %arg1[%arg4] [32] [1] : ... to memref<32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3] [32] [1] : ... to memref<32xf32, ...>
    linalg.matvec ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- Two `scf.for` tile loops stepping by 32: both of matvec's loops are tiled.
- `memref.subview` takes a 32-wide view of each operand: offsets, sizes, strides. Nothing is copied.
- Each slice follows its operand's indexing map. `vec`'s is `(i, j) -> (j)`, so its slice moves with the `j` loop (`%arg4`); `out`'s is `(i, j) -> (i)`, so it moves with the `i` loop (`%arg3`). `mat`'s is the identity: it moves with both.
- Inside, the original op on those slices. Still Linalg: tiling restructures the loops, it lowers nothing.

# Reference: `tile_linalg.mlir`, Step 3 of 3 — Applied to Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ... to memref<32x512xf32, ...>
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ... to memref<512x32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    linalg.matmul ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- The same schedule, unchanged: `i` and `j` tiled by 32, as for matvec.

# Reference: `tile_linalg.mlir`, Step 3 of 3 — Applied to Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ... to memref<32x512xf32, ...>
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ... to memref<512x32xf32, ...>
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ... to memref<32x32xf32, ...>
    linalg.matmul ins(%subview, %subview_2 : ...) outs(%subview_3 : ...)
  }
}
```

- The same schedule, unchanged: `i` and `j` tiled by 32, as for matvec.
- Matmul has a third loop, `k`, and `[32, 32]` leaves it alone. The slices of `a` and `b` are $32 \times 512$ and $512 \times 32$: the 512 is `k`, still untiled.

# Exercise 5 — Tiling Matmul's `k` Loop

Extend `session2/tile_linalg.mlir` so that matmul's `k` loop is tiled by 32 too.

**What is different from matvec:**

- Matmul has three loops; matvec has two.
- Two things that may help: `transform.structured.match ops{["linalg.matmul"]}` matches by op name, and a tile size of `0` leaves that loop untiled.

**Check after every save — round-trip it**:

```
!mlir-opt tile_linalg.mlir
```

- That checks the schedule is well-formed, not that it does what was intended. For that, the cells below the exercise apply it to matmul and check that its innermost op works on $32 \times 32$ slices.

10 minutes to write it — then we walk through the solution.

# Solution — `tile_linalg.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    %matmul = transform.structured.match ops{["linalg.matmul"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled_k, %k_loop = transform.structured.tile_using_for %matmul
      tile_sizes [0, 0, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    transform.yield
  }
}
```

# Solution — `tile_linalg.mlir`

```
module attributes {transform.with_named_sequence} {
  transform.named_sequence @__transform_main(
      %root: !transform.any_op {transform.readonly}) {
    %ops = transform.structured.match interface{LinalgOp} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled, %loops:2 = transform.structured.tile_using_for %ops
      tile_sizes [32, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op,
                                !transform.any_op)
    %matmul = transform.structured.match ops{["linalg.matmul"]} in %root
      : (!transform.any_op) -> !transform.any_op
    %tiled_k, %k_loop = transform.structured.tile_using_for %matmul
      tile_sizes [0, 0, 32]
      : (!transform.any_op) -> (!transform.any_op, !transform.any_op)
    transform.yield
  }
}
```

- Another option: `[32, 32, 32]` in the first step.

# Solution — Tiled Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ...
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ...
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ...
    ...
    scf.for %arg5 = %c0_4 to %c512 step %c32_5 {                          // k, by 32
      %subview_6 = memref.subview %subview[0, %arg5] [32, 32] [1, 1] : ...
      %subview_7 = memref.subview %subview_2[%arg5, 0] [32, 32] [1, 1] : ...
      %subview_8 = memref.subview %subview_3[0, 0] [32, 32] [1, 1] : ...
      linalg.matmul ins(%subview_6, %subview_7 : memref<32x32xf32, ...>, memref<32x32xf32, ...>)
                    outs(%subview_8 : memref<32x32xf32, ...>)
    }
  }
}
```

# Solution — Tiled Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ...
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ...
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ...
    ...
    scf.for %arg5 = %c0_4 to %c512 step %c32_5 {                            // k, by 32
      %subview_6 = memref.subview %subview[0, %arg5] [32, 32] [1, 1] : ...
      %subview_7 = memref.subview %subview_2[%arg5, 0] [32, 32] [1, 1] : ...
      %subview_8 = memref.subview %subview_3[0, 0] [32, 32] [1, 1] : ...
      linalg.matmul ins(%subview_6, %subview_7 : memref<32x32xf32, ...>, memref<32x32xf32, ...>)
                    outs(%subview_8 : memref<32x32xf32, ...>)
    }
  }
}
```

- A third tile loop, over `k`, stepping by 32.

# Solution — Tiled Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ...
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ...
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ...
    ...
    scf.for %arg5 = %c0_4 to %c512 step %c32_5 {                          // k, by 32
      %subview_6 = memref.subview %subview[0, %arg5] [32, 32] [1, 1] : ...
      %subview_7 = memref.subview %subview_2[%arg5, 0] [32, 32] [1, 1] : ...
      %subview_8 = memref.subview %subview_3[0, 0] [32, 32] [1, 1] : ...
      linalg.matmul ins(%subview_6, %subview_7 : memref<32x32xf32, ...>, memref<32x32xf32, ...>)
                    outs(%subview_8 : memref<32x32xf32, ...>)
    }
  }
}
```

- A third tile loop, over `k`, stepping by 32.
- It takes a subview of a subview: `%subview_6` is cut from `%subview`, the $32 \times 512$ strip of `a` that tiling `i` and `j` made, not from `a` itself. The same for `b`.

# Solution — Tiled Matmul

```
scf.for %arg3 = %c0 to %c128 step %c32 {                                    // i, by 32
  scf.for %arg4 = %c0_0 to %c256 step %c32_1 {                              // j, by 32
    %subview = memref.subview %arg0[%arg3, 0] [32, 512] [1, 1] : ...
    %subview_2 = memref.subview %arg1[0, %arg4] [512, 32] [1, 1] : ...
    %subview_3 = memref.subview %arg2[%arg3, %arg4] [32, 32] [1, 1] : ...
    ...
    scf.for %arg5 = %c0_4 to %c512 step %c32_5 {                          // k, by 32
      %subview_6 = memref.subview %subview[0, %arg5] [32, 32] [1, 1] : ...
      %subview_7 = memref.subview %subview_2[%arg5, 0] [32, 32] [1, 1] : ...
      %subview_8 = memref.subview %subview_3[0, 0] [32, 32] [1, 1] : ...
      linalg.matmul ins(%subview_6, %subview_7 : memref<32x32xf32, ...>, memref<32x32xf32, ...>)
                    outs(%subview_8 : memref<32x32xf32, ...>)
    }
  }
}
```

- A third tile loop, over `k`, stepping by 32.
- It takes a subview of a subview: `%subview_6` is cut from `%subview`, the $32 \times 512$ strip of `a` that tiling `i` and `j` made, not from `a` itself. The same for `b`.
- The check: every operand of the innermost `linalg.matmul` is $32 \times 32$.

# Lowering the Tiled Matmul

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
```
*then Session 1's CPU lowering pipeline*

- Tiling introduced `memref.subview`, which the step 1 pipeline never saw. Two more passes lower it, right after `convert-linalg-to-loops`.

# Lowering the Tiled Matmul

```
convert-linalg-to-loops
expand-strided-metadata     ← subviews become pointer and stride arithmetic ...
lower-affine
```

*then Session 1's CPU lowering pipeline*

- Tiling introduced `memref.subview`, which the step 1 pipeline never saw. Two more passes lower it, right after `convert-linalg-to-loops`.

# Lowering the Tiled Matmul

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine                ← ... which comes out as affine.apply: make it arith
```

*then Session 1's CPU lowering pipeline*

- Tiling introduced `memref.subview`, which the step 1 pipeline never saw. Two more passes lower it, right after `convert-linalg-to-loops`.

# Lowering the Tiled Matmul

```
convert-linalg-to-loops
expand-strided-metadata
lower-affine
```
*then Session 1's CPU lowering pipeline*

- Tiling introduced `memref.subview`, which the step 1 pipeline never saw. Two more passes lower it, right after `convert-linalg-to-loops`.
- Same numpy check as step 1:

```
OK
```

# Step 3 — Did Matmul Get Faster?

# Timing It

- Same computation, same numbers, same single thread: untiled against tiled.

# Timing It

- Same computation, same numbers, same single thread: untiled against tiled.
- `benchmark` keeps the fastest of five runs. Colab is a shared machine, so a slow run mostly measures other users.

# Timing It

- Same computation, same numbers, same single thread: untiled against tiled.
- `benchmark` keeps the fastest of five runs. Colab is a shared machine, so a slow run mostly measures other users.

| | |
|---|---|
| untiled | 9.0 ms |
| tiled | 0.8 ms |
| speedup | 10.9× |

one run on a desktop CPU — Colab's will differ

# Why So Much Faster? Read the Machine Code

- The usual answer is the cache. Not here: the three matrices together are under 1 MB, small enough to sit largely in cache untiled.

# Why So Much Faster? Read the Machine Code

- The usual answer is the cache. Not here: the three matrices together are under 1 MB, small enough to sit largely in cache untiled.
- What changed is the code LLVM generated. The next cell dumps the JIT's object code and counts the float multiplies and adds in `matmul`:

| | | |
|---|---|---|
| **scalar** | `mulss`, `addss`, ... | one float at a time |
| **packed** | `mulps`, `addps`, ... | 4, 8 or 16 floats at once, in `xmm`, `ymm`, `zmm` |

# Why So Much Faster? Read the Machine Code

- The usual answer is the cache. Not here: the three matrices together are under 1 MB, small enough to sit largely in cache untiled.
- What changed is the code LLVM generated. The next cell dumps the JIT's object code and counts the float multiplies and adds in `matmul`:

| | | |
|---|---|---|
| **scalar** | `mulss`, `addss`, ... | one float at a time |
| **packed** | `mulps`, `addps`, ... | 4, 8 or 16 floats at once, in `xmm`, `ymm`, `zmm` |

- Untiled: scalar only. Tiled: packed, 8 or 16 floats at once, depending on the CPU.

# Recap


linalg.matmul
tile_linalg.mlir
linalg.matmul
on 32×32 tiles
loops + Session 1's
pipeline
JIT + run
CPU, one thread

# Recap




- Linalg ops describe a computation — loops, maps, body — through one interface, `LinalgOp`.

# Recap




- Linalg ops describe a computation — loops, maps, body — through one interface, `LinalgOp`.
- A schedule written against that interface tiles any Linalg op. No C++, no rebuild.